

JAVA CODING FOR KIDS

Learn by doing 10 ready to type game in java

Marco Garagna

Copyright © 2025 by Marco Garagna. All Rights Reserved.

Table of Contents

1. Introduction to Java Programming for Kids

- Overview of Java Basics
- Setting Up Your Environment

2. Chapter 1: Hello World and Beyond

- "Hello Name" Game: A Simple Introduction to Java
- Modifying the Greeting

3. Chapter 2: Guessing Games

- "Guess My Number" Game: Introduction to Random Numbers
- "Guess My Color" Game: Using Arrays and Conditional Statements

4. Chapter 3: Interactive Games

- "Rock Paper Scissors" Game: Using Conditional Statements and Random Choices
- "Math Quiz" Game: Loops and Basic Arithmetic Operations

5. Chapter 4: Advanced Projects

- "Hangman" Game: Strings and Loops
- "Tic-Tac-Toe" Game: Grids and Conditional Logic

6. Chapter 5: Modifying and Creating Your Own Games

- Tips for Customizing Games
- Ideas for New Projects

1 Setting up for fun.

Learning to code is most fun when you're creating something you can play with! By typing and building these simple games, you're not just learning Java programming concepts—you're also developing problem-solving skills and understanding how code works in real-world applications.

The best way to learn is by doing. As you type each line of code, you'll see how it all comes together to create a working game. You'll learn about variables, loops, and conditional statements in a hands-on way that makes sense. Plus, you can modify the game to make it your own, which is a great way to experiment and learn even more!

So, get started and build your own games. You'll be amazed at how quickly you can create something cool and fun!

How to get started?

If you have already an IDE you can use that or you can type your games online and play without installing extra software.

Here is two simple path you can take to get ready:

1. Online Fun

Open a Web Browser:

Use a browser like Google Chrome, Firefox, or Safari.

Search for "Online Java Compiler":

Type these exact words in the search bar and press Enter.

Choose a Compiler:

Look at the search results and pick a compiler that looks easy to use. Some popular options include:

JDoodle: Known for its simplicity and ease of use.

TutorialsPoint: Offers a comprehensive environment with features like code execution and sharing.

OneCompiler: Provides a robust editor with features like auto-completion and error checking.

2. Use an IDE

If you have the possibility to set up an IDE (Integrated Development Environment) locally with the help of a parent or friend, you should try that! IDEs like IntelliJ IDEA or Eclipse make coding easier and provide powerful tools to help you learn Java.

However, if setting up an IDE feels too complicated right now, don't worry! You can start coding online using one of the compilers mentioned above. It's simple, quick, and perfect for beginners.

1 Hello World!

```
1  import java.util.Scanner;
2
3  public class HelloName {
4  public static void main(String[] args) {
5      Scanner scanner = new Scanner(System.in);
6
7      System.out.print("What is your name? ");
8      String name = scanner.nextLine();
9
10     System.out.println("Hello, " + name + "!");
11
12     scanner.close();
13 }
14 }
15
```

How Kids Can Modify This Game

1. **Add More Colors:** Add more colors to the colors array to make the game more challenging.
2. **Change the Number of Attempts:** Adjust the number of attempts allowed.
3. **Use Different Themes:** Instead of colors, use animals, fruits, or any other theme that kids enjoy.
4. **Add Hints:** Provide hints if the guess is close (e.g., "Your guess is a similar color").

```

import java.util.Scanner;

import java.util.Random;

public class CatchTheStar {

    public static void main(String[] args) {

        Scanner scanner = new Scanner(System.in);

        Random random = new Random();

        // Game settings - kids can modify these values
        int gameWidth = 10;    // Width of the game area
        int playerPosition = 5; // Starting position (middle)
        int score = 0;         // Starting score
        int rounds = 15;       // Number of rounds to play
        char playerChar = '^'; // Character representing the player
        char starChar = '*';   // Character representing stars
        char obstacleChar = 'X'; // Character representing obstacles

        System.out.println("=== CATCH THE STAR ===");
        System.out.println("Move left (L) or right (R) to catch stars (*) and avoid obstacles (X)");
        System.out.println("Stars = 10 points, Obstacles = -5 points");
        System.out.println("Press Enter to start the game!");
        scanner.nextLine();

        // Main game loop
        for (int round = 1; round <= rounds; round++) {

            // Create a falling object (either star or obstacle)
            int objectPosition = random.nextInt(gameWidth) + 1;
            boolean isStar = random.nextBoolean();
            char fallingObject = isStar ? starChar : obstacleChar;

            // Display the falling object

```

```

System.out.println("\nRound " + round + "/" + rounds + " | Score: " + score);
System.out.println();

// Show the falling object row
for (int i = 1; i <= gameWidth; i++) {
    System.out.print(i == objectPosition ? fallingObject : " ");
}
System.out.println();

// Show the player row
for (int i = 1; i <= gameWidth; i++) {
    System.out.print(i == playerPosition ? playerChar : " ");
}
System.out.println();

// Get player move
System.out.print("Move (L/R): ");
String move = scanner.nextLine().toUpperCase();

// Process player movement
if (move.equals("L") && playerPosition > 1) {
    playerPosition--;
} else if (move.equals("R") && playerPosition < gameWidth) {
    playerPosition++;
}

// Check if player caught the object
if (playerPosition == objectPosition) {
    if (isStar) {
        System.out.println("You caught a star! +10 points");
        score += 10;
    } else {
        System.out.println("Oh no! You hit an obstacle! -5 points");
    }
}

```

```

        score -= 5;
    }
    } else if (isStar) {
        System.out.println("You missed a star!");
    } else {
        System.out.println("You avoided an obstacle!");
    }
}

// Game over - display final score
System.out.println("\n=== GAME OVER ===");
System.out.println("Final score: " + score);

// Rate the performance
if (score >= 100) {
    System.out.println("Amazing! You're a star catcher champion!");
} else if (score >= 50) {
    System.out.println("Great job! You're getting good at this!");
} else if (score >= 0) {
    System.out.println("Not bad! Keep practicing!");
} else {
    System.out.println("Better luck next time!");
}

scanner.close();
}
}

```

What kids can easily modify:

- Change the game width to make it easier or harder
- Change the characters used for the player, stars, and obstacles
- Adjust the scoring system (how many points for stars/obstacles)
- Change the number of rounds
- Modify the final messages based on score
- Add new features (like different types of falling objects)

The game uses only basic Java concepts (variables, loops, conditionals, input/output), making it accessible for beginners. Kids can run this in any Java environment, including online IDEs if they don't have Java installed locally.

Formatting style (not part of the book)

-code snippet for ex. Page 5 I would like to achieve this look but not with a black square as background in printed version. How to do that?

The Picture now is just pasted in the document later needs to be attached from outside source.